

Token Limits, Rate Controls & Modality Restrictions

Chapter 08 · Episode 01 · Domain 2.3 — Resource Governance as Defense



Learning Objectives

1. Explain token ceilings, rate limits & modality restriction as one defense layer
2. Map volume attacks — DoS, cost amplification, model extraction — to the missing control
3. Recognize monitoring integration as what turns a limit into a defense

Three Volume-Based Attacks



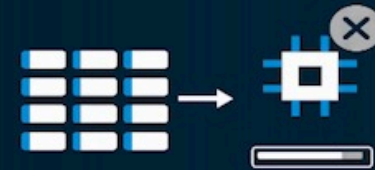
EXHAUSTION

Compute drained by volume
Degraded AI delays detection



AMPLIFICATION

Oversized context submissions
Economic, not informational



OPERATIONAL DoS

Unlimited request bursts
Overwhelms infrastructure

Set Maximum Token Ceilings

Caps context per inference cycle

Bounds cost AND manipulation surface

Unbounded windows expose earlier data

Limiting retention → data minimization



Rate Limiting

MECHANISMS

Per-user request thresholds

Time-based throttling

Burst detection

Daily/session quotas

WHY IT MATTERS

Primary defense vs model extraction

Reconstruction needs high query volume

Preventative + detective

Weak limits → scraping & recon

Modality Restriction

More modalities = more attack surface

Capability minimization at the gateway

Validate file format & size before parsing

Parsers/encoders carry their own bugs



Controls Need Monitoring to Count

Usage telemetry = the baseline

Cost anomaly detection flags abuse

Adaptive rate tightens in real time

A limit that never alerts = blind spot



1 Token Budget Enforcer

M
O
D

2 Burst Rate Detector

M
O
D

3 Modality Validator

M
O
D

HOW TO RUN

WINDOWS

```
python ep21_demo.py
```

MACOS/LINUX

```
python3 ep21_demo.py
```

TRY IT Lower the token ceiling to 100. Which legitimate queries break? What does this reveal about the usability/security tradeoff?

ep21_demo.py

pure Python 3.10+, no external libraries

Demo: Resource Governance Layer

Token-ceiling enforcement, rate limiting with burst detection, modality restriction, cost telemetry & adaptive alerts

Exam Tips



RESOURCE = SECURITY

Not just performance



MODALITY

Capability min at the gateway



RATE LIMITS

Primary anti-extraction defense



COST SPIKES

Detective control, not billing



Endpoint Protection & API Security

Chapter 08 · Episode 02 · Domain 2.3 — Inference APIs as Critical Infrastructure



Learning Objectives

1. Treat inference APIs as critical infrastructure, not auxiliary services
2. Pair authentication and authorization — both required, neither sufficient alone
3. Enforce TLS everywhere and wire endpoint telemetry into the SIEM

Authentication: Prove the Caller

MECHANISMS

API keys = basic ID, weak alone
Token-based: expiry, scope, revoke
OAuth / federated identity
Service-to-service auth internally

EXAM CONTENT

Eliminate anonymous access in prod
Even read-only enables extraction
Auth precedes every inference call
Inherit from enterprise IAM

Authorization: Scope What They Do

RBAC by role: analyst, admin, developer

ABAC adds context: dept, risk, geo

Scope inference / fine-tune / config /
retrieval separately

Inference priv modifying training = least-
privilege break



Encrypt Every Channel — TLS

TLS protects prompts & responses in transit

Certificate validation stops impersonation

Internal traffic needs TLS too

Trust-the-internal-network is a blind spot



Endpoint Monitoring = Visibility

Telemetry: request rate, tokens,
errors, params

Detect bursts, structured probing,
odd params

Correlate with identity → catch stolen creds

Alerts must trigger revoke + isolate,
feed SIEM



1 Auth Strength Evaluator

M
O
D

2 RBAC Scope Checker

M
O
D

3 Endpoint Exposure Map

M
O
D

HOW TO RUN

WINDOWS

```
python ep22_demo.py
```

MACOS/LINUX

```
python3 ep22_demo.py
```

TRY IT Add a new role with both inference and retraining privileges. Does the RBAC checker flag this as a least privilege violation?

ep22_demo.py

pure Python 3.10+, no external libraries

Demo: AI API Security Audit

Auth strength, authorization scoping, TLS enforcement, enumeration resistance
& monitoring integration checks

Exam Tips



CRITICAL INFRA

Treat like
financial/identity APIs



TLS INTERNAL

Internal networks aren't trusted



AUTHN + AUTHZ

Both required, different



SIEM

Standalone logs insufficient

Model Access Controls & Privilege Boundaries

Chapter 08 · Episode 03 · Domain 2.3 — Who Can Query, Modify, Retrain



Learning Objectives

1. Separate the three access boundaries: query, modify, retrain
2. Layer RBAC with ABAC scoping and environment boundaries
3. Protect model weights as IP and audit for privilege creep

RBAC and ABAC Scoping

RBAC — BY ROLE

Analyst → inference only

Engineer → retrain, hyperparams

Auditor → read-only logs

ABAC — BY CONTEXT

Dept, risk classification

Time of access, geo location

Retrain needs role + approved segment

Segment Dev, Staging & Production

Access to one environment $\neq$ access to another

Isolate experimental variants from prod

Deployment approval workflows
gate promotion

Config drift introduces boundary surprises



Model Weights Are IP Assets

Stolen weights → cloned models, exposed data

Tightly control export/download permissions

Encrypt artifacts at rest

Monitor abnormal downloads / large transfers



Audit for Privilege Creep

Periodic privilege audits =
only-what's-needed

Log retraining, config, deployment approvals

Alert on escalation, config change,
model swap

Configure-once-and-forget is the
failure mode



1 RBAC Scope Reviewer

M
O
D

2 Environment Boundary Check

M
O
D

3 Artifact Access Audit

M
O
D

HOW TO RUN

WINDOWS

```
python ep23_demo.py
```

MACOS/LINUX

```
python3 ep23_demo.py
```

TRY IT Grant an analyst role access to retraining. Does the RBAC reviewer immediately flag this as a least privilege violation?

ep23_demo.py

pure Python 3.10+, no external libraries

Demo: Model Access Control Audit

RBAC, ABAC scoping, environment segmentation, weight-artifact protection
& privilege-creep detection via log review

Exam Tips



×3

3 BOUNDARIES

Query / modify / retrain



ENVIRONMENTS

Dev compromise ≠ prod reach



RBAC vs ABAC

Role vs contextual attributes



WEIGHTS = IP

Export + encrypt + monitor

Data Access Controls in AI Systems

Chapter 08 · Episode 04 · Domain 2.3 — Data Boundaries Are the Perimeter



Learning Objectives

1. Treat data access and model access as independent control layers
2. Classify and segment datasets by sensitivity and role
3. Enforce retrieval-layer permissions before embeddings reach the model

Segment Datasets by Sensitivity

Classify: public → internal →
confidential → regulated

Fraud DB → financial security team only

Endpoint telemetry → SOC analysts only

Watch cross-dataset exposure &
bias amplification



Read vs Write Privilege Separation

READ — MANY

Visibility for evaluation

Monitor excessive inspection

Inspection spikes = recon

WRITE — FEW

Alters dataset contents

Unauthorized write = poisoning

Validated ingestion pipelines only

Retrieval Is Its Own Exposure Surface

Role-based retrieval: docs at/below clearance

Filter enforces BEFORE results reach model

Segment sensitive corpora from general KB

Inference-time overreach = retrieval governance fail



Audit the Data Layer

Log who accessed which datasets,
when, what role

Monitor unusual retrieval patterns
(indirect probing)

Correlate with identity: escalation,
geo anomalies

Review permissions each cycle —
stop privilege creep



1 Dataset Classifier

M
O
D

2 RBAC Access Checker

M
O
D

3 Retrieval Scope Filter

M
O
D

HOW TO RUN

WINDOWS

```
python ep24_demo.py
```

MACOS/LINUX

```
python3 ep24_demo.py
```

TRY IT Grant a general user access to a restricted corpus. Does the retrieval filter block the query before embeddings reach the model?

ep24_demo.py

pure Python 3.10+, no external libraries

Demo: Data Access Governance Audit

Dataset classification, RBAC, read/write separation, retrieval-layer permissions & access logging with anomaly detection

Exam Tips



TWO LAYERS

Data $\neq$ model — test both



READ vs WRITE

Least privilege on data



RAG SURFACE

Vector leakage is separate



OVERREACH

Retrieval fail, not model fail

CompTIA **SecAI+**

Agent Access Controls and Excessive Agency





Learning Objectives

1. Distinguish a model that generates output from an agent that takes action
2. Apply least privilege: scoped tools, allow-lists, time-bound tokens
3. Insert human-in-the-loop and prompt-to-action traceability

Agent Least Privilege

Grant only tools the role strictly needs
Scoped tool permissions per external system
Command allow-lists, not broad write access
Time-bound tokens expire → short exposure window

ONE AGENT, SCOPED TOOLS



TOOLS

ROLE-SCOPED



COMMANDS

ALLOW-LIST



TOKENS

TIME-BOUND



BROAD WRITE

DENIED



TASK-SCOPED ACCESS

The agent gets what its task needs — nothing else, not for long.

Human-in-the-Loop for High-Impact

REQUIRE APPROVAL

Account suspension

Firewall rule changes

Financial approvals

OVERSIGHT DESIGN

Approval checkpoints = friction

Escalation thresholds

Dual authorization

Trace Every Action to Its Prompt

Prompt → action linkage, not generic logs

Anomaly detection on execution sequences

Correlate prompt history with action logs

Rapid rollback reverses unauthorized changes



Structural Containment

Sandbox execution isolates from production
Environment segmentation: test $\neq$ prod
Progressive escalation: temporary, approved
Defense in depth — no single control suffices

CONTAIN BY STRUCTURE

SANDBOX



AGENT EXECUTES HERE

ISOLATED



ESCAPE · DENIED



APPROVED · TIMED



PRODUCTION

LIVE

No single control suffices — isolation is built into the structure.

1 Tool Permission Checker

M
O
D

2 HITL Approval Gate

M
O
D

3 Action Trace Logger

M
O
D

HOW TO RUN

WINDOWS

```
python ep25_demo.py
```

MACOS/LINUX

```
python3 ep25_demo.py
```

TRY IT Lower the approval threshold to require HITL for medium-impact actions. How does this change the autonomy/oversight tradeoff?

ep25_demo.py

pure Python 3.10+, no external libraries

Demo: Agent Access Control Framework

Scoped permissions, command allow-lists, human-in-the-loop thresholds, time-bound tokens & action logging with rollback

Exam Tips



MODEL vs AGENT

Action → stricter controls



LEAST PRIV

Scope, allow-list, TTL tokens



EXCESS AGENCY

Root of unintended changes



TRACEABILITY

Action → initiating prompt

CompTIA **SecAI+**

Network and Infrastructure Access Controls





Learning Objectives

1. Segment the AI ecosystem to contain lateral movement
2. Apply zero trust to internal east-west traffic
3. Isolate training vs inference infra and govern config drift

Segmentation Contains Blast Radius

ZONES

Dev / staging / production
Training vs inference clusters
Storage environments

WHY

Compromised API $\neq$ training data reach
Minor endpoint $\rightarrow$ full model manipulation
= insufficient segmentation

Zero Trust for East-West Traffic

Service-to-service auth — verify, never assume

Mutual TLS: client and server authenticate

Short-lived credentials limit persistence

Every connection: authenticated, authorized, logged

SERVICE TO SERVICE, VERIFIED



EVERY CONNECTION

AUTHENTICATED

AUTHORIZED

LOGGED

UNVERIFIED SERVICE

—X REFUSED

Being inside the network proves nothing — verify every hop.

Isolate Training vs Inference

Never share compute without isolation

Dedicated clusters reduce shared-resource risk

GPU memory can leak across workloads

Dev experiment affecting prod = separation fail



Micro-Segmentation + Visibility

Enforce policy between individual workloads

Only inference service queries
embedding store

Only training service reaches raw
dataset storage

Baseline traffic; unexpected transfers
= compromise



Govern Config Drift with IaC

Infrastructure as code = version-controlled topology

Change management: approve before apply

Rollback pipelines at the infra layer

Auditable, attributable config changes



1 Zone Boundary Checker

M
O
D

2 mTLS Validator

M
O
D

3 IaC Drift Detector

M
O
D

HOW TO RUN

WINDOWS

```
python ep26_demo.py
```

MACOS/LINUX

```
python3 ep26_demo.py
```

TRY IT Remove the boundary between training and inference zones in the simulator. What lateral movement paths become available to an attacker?

ep26_demo.py

pure Python 3.10+, no external libraries

Demo: AI Infrastructure Segmentation

Zone definitions, service-to-service auth, east-west policy enforcement, micro-segmentation rules & IaC change audit

Exam Tips



EAST-WEST

Under-monitored blind spot



TRAIN vs INFER

Isolate or separation fails



ZERO TRUST

Applies internally too



IaC + CHANGE

Answer for config drift

CompTIA **SecAI+**

Encryption Requirements for AI Systems





Learning Objectives

1. Map the three encryption states to their exam triggers
2. Pair strong ciphers with disciplined key governance
3. Extend encryption in use to protect active computation

Encryption at Rest

Encrypt datasets, corpora, vector DBs, weights

Volume-level + application-level layers

Isolate keys from the encrypted data

Integrate enterprise KMS:
rotation, revocation

EVERYTHING AT REST, ENCRYPTED



DATASETS



CORPORA



VECTOR DB



WEIGHTS



KEYS LIVE APART — ONE KMS



TWO LAYERS DEEP

A stolen volume without its key yields nothing.

Encryption in Use

Data is exposed in memory during compute

Confidential computing = trusted execution env

Isolates memory in multi-tenant cloud

Homomorphic: compute on encrypted data



Key Governance Ties It Together

Rotation reduces long-term key exposure

Revocation invalidates compromised
keys now

Log cryptographic operations for forensics

Document standards for regulated data



1 At-Rest Coverage Check

M
O
D

2 TLS Configuration Audit

M
O
D

3 Key Governance Review

M
O
D

HOW TO RUN

WINDOWS

```
python ep27_demo.py
```

MACOS/LINUX

```
python3 ep27_demo.py
```

TRY IT Simulate key co-location by placing the encryption key in the same directory as the dataset. Does the audit flag this as a critical misconfiguration?

ep27_demo.py

pure Python 3.10+, no external libraries

Demo: Encryption Posture Audit

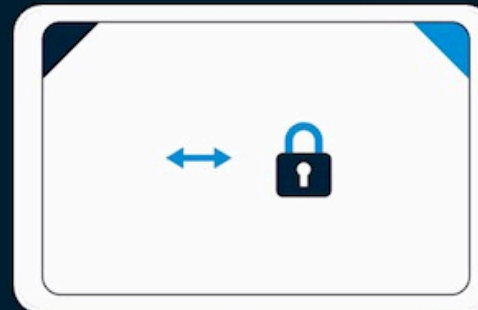
At rest (datasets, weights, vector DBs), in transit (TLS), in use (confidential computing) + key-governance checks

Exam Tips



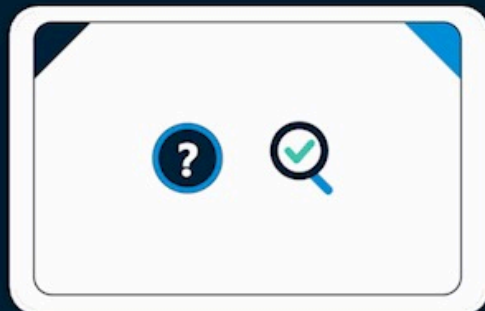
THREE STATES

Rest / transit / use



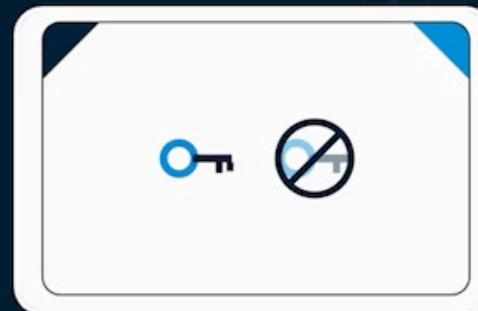
MUTUAL TLS

Internal svc-to-svc expectation



MATCH SCENARIO

Symptom → correct state



KEY GOVERNANCE

Rotate, revoke, log